I. Text Manipulation in Python

Target	Function	Python shell	Outputs
Print a **Text**	`print()`	`Text = "Hello World!"` `print(Text)`	Hello World!
Return the first character of a **Text**		`Text = "Hello World!"` `print(Text[0])`	H
Return a substring from a **Text**		`Text = "Hello World!"` `print(Text[3:8])`	lo wo
Return a substring from a **Text** starting at the beginning		`Text = "Hello World!"` `print(Text[:5])`	Hello
Return a substring from a given position to the end of a **Text**		`Text = "Hello World!"` `print(Text[6:])`	World!
Return the length of a **Text**	`len()`	`Text = "Hello World!"` `print(len(Text))`	12
Return the number of occurrences of a specific character in a **Text**	`count()`	`Text = "Hello World!"` `print(Text.count("o"))`	2
Split a **Text**	`split()`	`Text = "Hello World!"` `print(Text.split())`	['Hello', 'World!']

Return the starting index of a substring in a **Text**	`find()`	`Text = "Hello World!"`
		`print(Text.find("W"))`
		`print(Text.find("o"))`
		`print(Text.find("SII"))`
			6
			4
			-1

Return the starting index of a substring in a **Text**	`index()`	`Text = "Hello World!"`
		`print(Text.index("H"))`
		`print(Text.index("d!"))`
			1
			10

Convert all characters in a **Text** to uppercase	`upper()`	`Text = "Hello World!"` `print(Text.upper())`	HELLO WORLD!
Convert all characters in a **Text** to lowercase	`lower()`	`Text = "Hello World!"` `print(Text.lower())`	hello world!
Capitalize the first letter of every word in a **Text**	`title()`	`Text = "hello world!"` `print(Text.title())`	Hello World!
Capitalize only the first letter of a **Text**	`capitalize()`	`Text = "hello world!"` `print(Text.capitalize())`	Hello world!
Change the case of each letter in a **Text**	`swapcase()`	`Text = "Hello World!"` `print(Text.swapcase())`	hELLO wORLD

Target	Function	Python shell	Outputs
Add a substring to a **Text**		`Text = "Hello World!"` `print(Text+ " :)")`	Hello World! :)
Multiply a **Text**		`Text = "Hello World! "` `print(Text*4)`	Hello World! Hello World! Hello World! Hello World!
Reverse a **Text**	`reversed()`	`Text = "Hello World!"` `print(''.join(reversed(Text)))`	!dlroW olleH
Add a whitespace between every character in a **Text**	`join()`	`Text = "Hello World!"` `print(" ".join(Text))`	H e l l o W o r l d !
Check if all characters in a **Text** are alphanumeric	`isalnum()`	`Text = "Hello World!"` `print(Text.isalnum())`	False
Check if all characters in a **Text** are alphabetic	`isalpha()`	`Text = "Hello World!"` `print(Text.isalpha())`	False
Check if all characters in a **Text** are digits	`isdigit()`	`Text = "Hello World!"` `print(Text.isdigit())`	False
Check if the first letter of every word in a **Text** is capitalized	`istitle()`	`Text = "Hello World!"` `print(Text.istitle())`	True
Check if a **Text** is in uppercase	`isupper()`	`Text = "Hello World!"` `print(Text.isupper())`	False
Check if a **Text** is in lowercase	`islower()`	`Text = "Hello World!"` `print(Text.lower())`	False
Check if a **Text** starts with a specific character	`startswith()`	`Text = "Hello World!"` `print(Text.startswith('H'))`	True
Check if a **Text** ends with a specific character	`endswith()`	`Text = "Hello World!"` `print(Text.endswith('!'))`	True
Replace a substring in a **Text**	`repalce()`	`Text = "Hello World!"` `print(Text.replace("Wo","SII"))`	Hello SIIrld
Remove leading and trailing whitespace or blank lines in a **Text**	`strip()`	`Text = "   Hello World!  "` `print(Text.replace())`	Hello World!

II. Text Extraction

Extracting Text from a TXT file

```
txt = open("Name.txt", "r")
for line in txt:
    print(line)
            txt.close()
```

Extracting text from a DOC file

```
#Install library python-docx==1.2.0

import docx
from docx import Document

doc = open ("Name.docx", "rb")
document = Document(doc)

#Extracting text from a document word
doc_text = ""
for para in document.paragraphs:
    doc_text+= para.text

print(doc_text)
doc.close()
```

Extracting Text from a PDF file

```
#Install library PyPDF2==3.0.1
#PyPDF2: PDF manipulation library
from PyPDF2 import PdfReader

#Open PDF file
pdf = open("Name.pdf", "rb")

#PdfReader: read and interpret PDF files
#Creating PDF reader object
PDFreader = PdfReader(pdf)

# PDFreader.pages: list of the document's pages
# Fetching the number of pages in the PDF
num_pages = len(PDFreader.pages)
print(num_pages)

if num_pages > 0:
      #Selecting page 0
      page = PDFreader.pages[0]

      #extract_text(): extract text from a page
      text = page.extract_text()
      print(text)

 pdf.close()
```

Extracting text from a SCANNED document

```
#Install library pytesseract==0.3.10
#pytesseract: OCR manipulation library

#Install library opencv-python==4.10.0
#opencv-python(cv2): image manipulation library

import cv2
from pytesseract import image_to_string

filename = "Name.png"

#imeread(): read an image
img = cv2.imread(filename)

#image_to_text(): extract text from an image
text = image_to_string(img,lang ='eng')
print (text)
```

Extracting Text from a HTML page (web scraping)

```
#Install library BeautifulSoup4==4.13.5
#BeautifulSoup: parse HTML (analyze and extract data from web pages.

from bs4 import BeautifulSoup
from urllib.request import urlopen

myurl = "https://link.springer.com/article/10.1007/s12065-022-00794-z"

html = urlopen(myurl).read()

#Parse and convert the entire HTML page into a tree structure
soup = BeautifulSoup(html, "html.parser")

#Searches inside the parsed HTML:
#for a <div> tag that has the class c-article-body
text = soup.find("div",{"class":"c-article-body"})

#Extract all readable text inside the <div> (removes the HTML tags)
print (text.get_text())

abstract = soup.find("div",{"class":"c-article-section__content"})
print(abstract.get_text())
```

III. Text Preprocessing

NLTK, **spaCy** and **TextBlob** are commonly used **Natural Language Processing (NLP)** libraries in Python for text processing.

III. 1. Installation

1| NLTK library installation

NLTK	**NLTK** (Natural Language Toolkit) is a **NLP library** in Python It provides tools for **text preprocessing**, such as tokenization, stemming and lemmatization.

How to Use NLTK library for Text Processing

Steps	From	Commands
Step 1 — Install the NLTK Library	Terminal	`#Recommended: NLTK==3.9.2 (requires Python <= 3.11)` `pip install nltk`
Step 2 — Launch the NLTK Linguistic Data Downloader	Python Shell	`import nltk` `nltk.download()`
Step 3 — Download and Install `punkt` model Needed for word and sentence segmentation		
Step 4 — Download and Install `punkt_tab` model Needed for word and sentence segmentation		
Step 5 — Download and Install `stopwords` corpus Needed for stop word removal		

2| spaCy library installation

spaCy	**spaCy** is a **NLP library** in Python. It offers powerful functionalities, including: **Part-of-Speech (POS) tagging**: identifies the **grammatical category** of **each word in a text**, and **Named Entity Recognition (NER)**: detects and classifies proper nouns such as names of people, organizations, etc. **spaCy Language Model (LM)** allow performing **NLP tasks** such as **Tokenization**, **POS tagging** and **NER**.

How to Use spaCy library for Text Processing

Steps	From	Commands
Step 1 — Install the spaCy Library	**Terminal**	`#Recommended: spaCy==3.7.2 (requires Python <= 3.11)` `pip install spacy`
Step 2 — Download the English Language Model	**Terminal**	`spacy download en_core_web_sm`
Step 3 — Load the English Language Model	**Python Shell**	`import spacy` `nlp = spacy.load('en_core_web_sm')`
Step 4 — Process Text Using the spaCy NLP Pipeline[1]	**Python Shell**	`doc = nlp('U.S.T.H.B. is a University in Algeria.')`

3| TextBlob library installation

TextBlob	**TextBlob is a Python library for processing textual data that provides simple APIs for common NLP tasks such as tokenization, part-of-speech tagging, sentiment analysis, and text classification.**

How to Use spaCy library for Text Processing

Steps	From	Commands
Step 1 — Install the TextBlob Library	**Terminal**	`#Recommended: TextBlob==0.19.0 (Python <= 3.11)` `pip install textblob`

[1] The NLP pipeline in spaCy performs a sequence of text processing steps, starting with word segmentation (tokenization).

III.2. Text Preprocessing with NLTK

1| Word Segmentation (Tokenization)

Description	Python Shell	Outputs

Split the text into meaningful tokens using **word_tokenize**	`import nltk`
	`Text = "U.S.T.H.B. is a University in Algeria."`
	`Text = Text.lower() #Lowercasing before tokenizing`
	`Text = nltk.word_tokenize(Text)`
	`for token in Text:`
	`  print(token)`
		u.s.t.h.b
		.
		is
		a
		university
		in
		algeria
		.

2| Sentence segmentation

Description	Python Shell	Outputs

Split the text into multiple sentences using **sent_tokenize**	`import nltk`
	`Text = "Do you know? U.S.T.H.B. is a University."`
	`Text = Text.lower()`
	`Text = nltk.sent_tokenize(Text)`
	`for sent in Text:`
	`  print(sent)`
		do you know?
		u.s.t.h.b.
		is a university.

3| Word and Sentence segmentation using Regexp

Description	Python Shell	Outputs

Split text into tokens or sentences, depending on the desired criteria, using **regexp**	`import nltk`
	
	`Text = "U.S.T.H.B. is a University in Algeria."`
	`ExpReg = nltk.RegexpTokenizer('\w+') #w > [a-zA-Z0-9_]`
	`Text = nltk.ExpReg.tokenize(Text)`
	`for sent in Text:`
	`  print(sent)`
	
	`Text = " U.S.T.H.B. is a University in Algeria."`
	`ExpReg = nltk.RegexpTokenizer('(?:[A-Z]\.)+|\w+')`
	`Text = nltk.ExpReg.tokenize(Text)`
	`for sent in Text:`
	`  print(sent)`
		U
		S
		T
		H
		B
		is
		a
		University
		in
		Algeria
		
		U.S.T.H.B.
		is
		a
		University
		in
		Algeria

4| Word Normalization

Two popular techniques for achieving word **normalization** are **stemming** and **lemmatization**. Stemming refers to the process of removing suffixes and reducing a word to its base or root form so that all its variants can be represented by the same form. Lemmatization, on the other hand, maps all the different forms of a word to its dictionary base form, or lemma. While this may seem similar to stemming, lemmatization requires deeper linguistic knowledge and modeling.

Description	Python Shell	Outputs

Word **Stemming**
(Porter stemmer)
	`import nltk`
	`stemmer = nltk.PorterStemmer()`
	`Text = "U.S.T.H.B. is a University in Algeria."`
	`Text = Text.lower()`
	`Text = nltk.word_tokenize(Text)`
	`for token in Text:`
	`  print(token, stemmer.stem(token))`
		u.s.t.h.b **u.s.t.h.b**
		. .
		is **is**
		a **a**
		university **univers**
		in **in**
		algeria **algeria**
		. .

Description	Python Shell	Outputs

Word **Stemming**
(Lancaster stemmer)
	`import nltk`
	`stemmer = nltk.LancasterStemmer()`
	`Text = "U.S.T.H.B. is a University in Algeria."`
	`Text = Text.lower()`
	`Text = nltk.word_tokenize(Text)`
	`for token in Text:`
	`  print(token, stemmer.stem(token))`
		u.s.t.h.b **u.s.t.h.b**
		. .
		is **is**
		a **a**
		university **univers**
		in **in**
		algeria **alger**
		. .

Description	Python Shell	Outputs

Word **Stemming**
(Snowball stemmer)
	`import nltk`
	`stemmer = nltk.SnowballStemmer("english")`
	`Text = "U.S.T.H.B. is a University in Algeria."`
	`Text = Text.lower()`
	`Text = nltk.word_tokenize(Text)`
	`for token in Text:`
	`  print(token, stemmer.stem(token))`
		u.s.t.h.b **u.s.t.h.b**
		. .
		is **is**
		a **a**
		university **univers**
		in **in**
		algeria **algeria**
		. .

Description	Python Shell	Outputs

Word **Lemmatization**	`import nltk`
	`lemma = nltk.WordNetLemmatizer()`
	`Text = "U.S.T.H.B. is a University in Algeria."`
	`Text = Text.lower()`
	`Text = nltk.word_tokenize(Text)`
	
	`for token in Text:`
	`  # Assume the POS tag for all tokens is "v" (verb).`
	`  # Use nltk.pos_tag() to get real POS tags.`
	`  # POS tagging assigns each token a grammatical`
	`  # category such as noun, verb, or adjective).`
	`  print(token, lemma.lemmatize(token, pos="v"))`
		u.s.t.h.b **u.s.t.h.b**
		. .
		is **be**
		a **a**
		university **university**
		in **in**
		algeria **algeria**
		. .

5| Stop Word Removal

In NLP, the words do not always provide useful insights are called stop words. There is no universal stop words list for each language. Usually, it is a list of the most common words in the language, such as of, the, want, to, and have.

Description	Python Shell	Outputs

Check if the token is a stop word or not.	`import nltk`
	`stop = nltk.corpus.stopwords.words('english')`
	`Text = "U.S.T.H.B. is a University in Algeria."`
	`Text = Text.lower()`
	`Text = nltk.word_tokenize(Text)`
	
	`for token in Text:`
	`  if token in stop:`
	`  print(token, True)`
	`  else:`
	`  print(token, False)`
		u.s.t.h.b **False**
		. **False**
		is **True**
		a **True**
		university **False**
		in **True**
		algeria **False**
		. **False**

III.2. Text Preprocessing and with spaCy

1| Word Segmentation (Tokenization)

Description	Python Shell	Outputs

The **first step** in the **spaCy NLP pipeline** is **Tokenization**. spaCy uses **rule-based tokenization**, relying on **language-specific rules** to extract tokens.	`import spacy`
	`nlp = spacy.load('en_core_web_sm')`
	`Text = nlp('U.S.T.H.B. is a University in Algeria.')`
	`for token in Text:`
	`  print(token.text)`
		U.S.T.H.B.
		is
		a
		University
		in
		Algeria
		.

2| Part-of-Speech (POS) tagging

Description	Python Shell	Outputs

Another step in the **spaCy NLP** pipeline is **POS tagging**. This process is **model-based**, meaning it relies on statistical, machine learning (ML), or deep learning (DL) models to predict the correct tags.	`import spacy`
	`nlp = spacy.load('en_core_web_sm')`
	`Text = nlp('U.S.T.H.B. is a University in Algeria.')`
	`for token in Text:`
	`  print(token.text, token.pos_)`
		U.S.T.H.B. **PROPN**
		is **AUX**
		a **DET**
		University **PROPN**
		in **ADP**
		Algeria **PROPN**
		. **PUNCT**

3| Named Entity Recognition (NER)

Description	Python Shell	Outputs

Another key step in the **spaCy NLP pipeline** is **NER tagging**, which identifies and classifies named entities in text (such as people, organizations, locations). This process is **model-based**, meaning it relies on statistical, ML, or DL models to predict the correct tags.	`import spacy`
	`nlp = spacy.load('en_core_web_sm')`
	`Text = nlp('U.S.T.H.B. is a University in Algeria.')`
	`for ent in Text.ents:`
	`  print(ent.text, ent.label_)`
		U.S.T.H.B. **GPE**
		Algeria **GPE**

4| Sentence segmentation

Description	Python Shell	Outputs

Split the text into multiple sentences.	`import spacy`
	`nlp = spacy.load('en_core_web_sm')`
	`Text = nlp('Do you know? U.S.T.H.B. is a University.')`
	`for sent in Text.sents:`
	`  print(sent)`
		Do you know?
		U.S.T.H.B. is a University.

5| Word Normalization

Description	Python Shell	Outputs

Word **lemmatization**	`import spacy`
	`nlp = spacy.load('en_core_web_sm')`
	`doc = nlp('U.S.T.H.B. is a University in Algeria.')`
	`for token in doc:`
	`  print(token.text, token.lemma_)`
		U.S.T.H.B. U.S.T.H.B.
		is be
		a a
		University University
		in in
		Algeria Algeria

2| Stop Word Removal

Description	Python Shell	Outputs

Add **custom stop words** to the standard list.	`import spacy`
	`nlp = spacy.load('en_core_web_sm')`
	`my_stop_words = ['say', 'be', 'said', 'says']`
	`for stopword in my_stop_words:`
	`  token = nlp.vocab[stopword]`
	`  token.is_stop = True`
	`doc = nlp("say be said says")`
	`for token in doc:`
	`  print(token.text, token.is_stop)`
		say True
		be True
		said True
		says True

Description	Python Shell	Outputs

Add **a stop words** to the list of stop word.	`import spacy`
	`nlp = spacy.load('en_core_web_sm')`
	
	`stop_words_list = spacy.lang.en.stop_words.STOP_WORDS`
	`stop_words_list.add('saying')`	

Description	Python Shell	Outputs

Check if the token is a stop word or not using attribute **is_stop**	`import spacy`
	`nlp = spacy.load('en_core_web_sm')`
	`doc = nlp('U.S.T.H.B. is a University in Algeria.')`
	`for token in doc:`
	`  print(token.text, token.is_stop)`
		U.S.T.H.B. **False**
		is **True**
		a **True**
		University **False**
		in **True**
		Algeria **False**
		. **False**

Overview of spaCy's Built-in Token Attributes and Linguistic Features

Attribute	Description	Attribute	Description
`lower_`	Lowercase form of the **Token**	`is_title`	If the first letter of the **Token** is capitalized
`upper_`	Uppercase form of the **Token**	`is_punct`	**Token** is punctuation
`is_alpha`	**Token** consists of alphanumeric chars	`is_space`	**Token** is whitespaces
`is_digit`	**Token** consists of digits	`like_num`	**Token** resembles a number
`is_lower`	**Token** text is in lowercase	`like_url`	**Token** resembles an URL
`is_upper`	**Token** text is in uppercase	`like_email`	**Token** resembles an email

POS Tags			NER Tags	
Tag	**Full name**	**Example**	**Label**	**Description**
`ADJ`	Adjective	happy, new	`ORG`	Organizations (companies, agencies, institutions, etc.)
`ADP`	Adposition	in, to, during	`GPE`	Geopolitical entities (countries, cities, states)
`ADV`	Adverb	quickly, never	`LOC`	Non-GPE locations (mountain ranges, bodies of water, etc.)
`AUX`	Auxiliary verb	is, have, do	`PRODUCT`	Objects, vehicles, foods, etc. (not services)
`CCONJ`	Coordinating conjunction	and, but, or	`EVENT`	Named events (wars, sports events, natural disasters)
`DET`	Determiner	the, a, this	`WORK_OF_ART`	Titles of books, songs, artworks, etc.
`INTJ`	Interjection	oh, wow, hey	`LAW`	Named laws or legal documents
`NOUN`	Noun	dog, house, computer	`LANGUAGE`	Named languages
`NUM`	Numeral	one, 100, third	`DATE`	Dates or periods (e.g., January, 2020, last year)
`PART`	Particle	to, not	`TIME`	Times smaller than a day (e.g., 2 p.m., morning)
`PRON`	Pronoun	he, she, they, it	`PERCENT`	Percentage values (e.g., 50%)
`PROPN`	Proper noun	John, London, Microsoft	`MONEY`	Monetary values (e.g., $5, €100)
`PUNCT`	Punctuation	., ?, !	`QUANTITY`	Measurements (e.g., 10 km, 5 liters)
`SCONJ`	Subordinating conjunction	because, although	`ORDINAL`	Ordinal numbers (e.g., first, second)
`SYM`	Symbol	$, %, ©	`CARDINAL`	Cardinal numbers (e.g., one, 1000)
`VERB`	Verb	run, eat, think		
`X`	Other	foreign words, errors		

III.3. Text Preprocessing with TextBlob

1| Word Segmentation (Tokenization)

Description	Python Shell	Outputs

Split the text into meaningful tokens using **TextBlob().words**	`import textblob`
	`Text = "U.S.T.H.B. is a University in Algeria."`
	`Text = Text.lower() #Lowercasing before tokenizing`
	
	`Text = textblob.TextBlob(Text).words`
	`for token in Text:`
	`  print(token)`
		u.s.t.h.b
		is
		a
		university
		in
		algeria

2| Sentence segmentation

Description	Python Shell	Outputs

Split the text into multiple sentences using **TextBlob().sentences**	`import textblob`
	`Text = "Do you know? U.S.T.H.B. is a University."`
	`Text = Text.lower()`
	`Text = textblob.TextBlob(Text).sentences`
	`for sent in Text:`
	`  print(sent)`
		do you know?
		u.s.t.h.b.
		is a university.

3| Word Normalization

Description	Python Shell	Outputs

Word **Lemmatization**	`import textblob`
	`Text = "U.S.T.H.B. is a University in Algeria."`
	`Text = Text.lower()`
	`Text = textblob.TextBlob(Text).words`
	
	`for token in Text:`
	`  # Assume the POS tag for all tokens is "v" (verb).`
	`  # Use attribute .pos_tags to get real POS tags.`
	`  Lemma = textblob.Word(token).lemmatize(pos="v")`
	`  print(token, Lemma)`
		u.s.t.h.b **u.s.t.h.b**
		is **be**
		a **a**
		university **university**
		in **in**
		algeria **algerias**

4| Spelling corrections

Description	Python Shell	Outputs

Correct spelling errors using **correct()**	`import textblob`
	`Text = "Doo youu kwnow? U.S.T.H.B. ies a University."`
	`Text = Text.lower()`
	`print(Text)`
	
	`Text = textblob.TextBlob(Text).correct()`
	`print(Text)`
		`Doo youu kwnow? U.S.T.H.B. ies a Univercity."`
		
		`do you know? u.s.t.h.b. is a university.`

IV. Distance Metrics

Many NLP applications involve tasks such as computing the similarity between two pieces of text. This can be done at different levels: word, phrase, sentence, or document. The goal may be to identify either syntactic or semantic similarity. Similarity measures can also be applied in various contexts; one such context is character-level similarity, which examines how different two strings are based on their characters. Popular approaches for measuring this difference include the Levenshtein edit distance and Jaro similarity methods. **Textdistance** is a popular Python library for computing such measures.

Description	Python Shell	Outputs

Levenshtein Edit Distance
It measures the minimum number of operations required to transform one string into another. The allowed operations include insertion, deletion, or substitution of a character.
	`import textdistance`
	
	`print(textdistance.levenshtein('Algeria', 'Algiers'))`
		3

Jaro similarity
It measures string similarity based on matching characters and their order, yielding a score from 0 (no match) to 1 (exact match).
	`import textdistance`
	
	`print(textdistance.jaro('Algeria', 'Algiers'))`
		0.8492063492063492

V. Text Visualization

Description	Python Shell	Outputs

WordCloud is the most popular Python library for representing text. The WordCloud library allows the generation of visualizations from a body of text, where the frequency of words or phrases is reflected by their size and opacity in the plot.	`# Import the matplotlib library for creating plots`
	`import matplotlib`
	`# Use the Qt5 graphical interface`
	`matplotlib.use('Qt5Agg')`
	`# pyplot provides functions for plotting`
	`import matplotlib.pyplot as plt`
	`# To generate a word cloud visualization from text`
	`from wordcloud import WordCloud`
	`Text = "U.S.T.H.B. is a University in Algeria."`
	`wordcloud = WordCloud(width=800, height=400, background_color='white').generate(Text)`
	`# Display the word cloud image`
	`plt.imshow(wordcloud.to_array(), interpolation='bilinear')`
	`# Remove the axis labels for a cleaner visualization`
	`plt.axis('off')`
	`# Display the plot`
	`plt.show()`
		

References

[1] Bhargav Srinivasa-Desikan. Natural Language Processing and Computational Linguistics (2018)

[2] Delip Rao and Brian McMahan. Natural Language Processing with PyTorch - Build Intelligent Language Applications Using Deep Learning (2019)

[3] Jyotika Singh. Natural Language Processing in the Real World. Text Processing, Analytics, and Classification (2023)

[4] Rosario Moscato, et al. Natural Language Processing Cookbook - Step-by-step practical solution for unlocking the power of natural language processing potential (2025)

[5] Sowmya Vajjala, et al. Practical Natural Language Processing - A Comprehensive Guide to Building Real-World NLP Systems (2020)

Exercise:

Dataset

We will use the **BBC News dataset**, which contains **2225 news articles** published between 2004 and 2005, divided into five categories: **Business, Entertainment, Politics, Sport and Tech.**

Dataset link: [BBC News – Full Text Dataset]

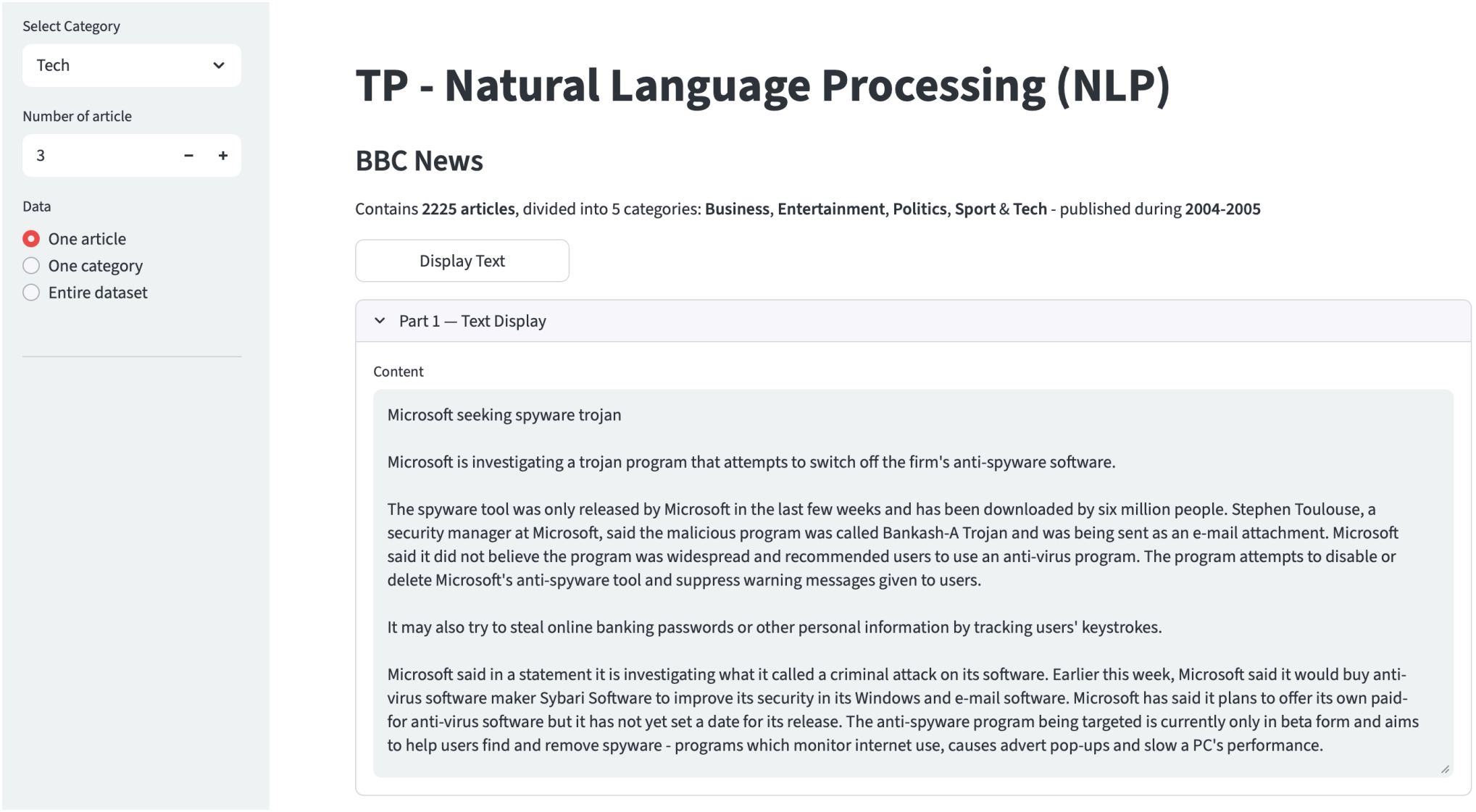
I. Text Display

> Implement the necessary functions to display the text at three different levels:

. **One article:** Display a selected article.

. **One category**: Display all articles belonging to a selected category.

. **Entire dataset:** Display all articles in the dataset.


Select Category
Tech
Number of article
3
Data
One article
One category
Entire dataset
TP - Natural Language Processing (NLP)
BBC News
Contains 2225 articles, divided into 5 categories: Business, Entertainment, Politics, Sport & Tech - published during 2004-2005
Display Text
Part 1 — Text Display
Content
Microsoft seeking spyware trojan
Microsoft is investigating a trojan program that attempts to switch off the firm's anti-spyware software.
The spyware tool was only released by Microsoft in the last few weeks and has been downloaded by six million people. Stephen Toulouse, a security manager at Microsoft, said the malicious program was called Bankash-A Trojan and was being sent as an e-mail attachment. Microsoft said it did not believe the program was widespread and recommended users to use an anti-virus program. The program attempts to disable or delete Microsoft's anti-spyware tool and suppress warning messages given to users.
It may also try to steal online banking passwords or other personal information by tracking users' keystrokes.
Microsoft said in a statement it is investigating what it called a criminal attack on its software. Earlier this week, Microsoft said it would buy anti-virus software maker Sybari Software to improve its security in its Windows and e-mail software. Microsoft has said it plans to offer its own paid-for anti-virus software but it has not yet set a date for its release. The anti-spyware program being targeted is currently only in beta form and aims to help users find and remove spyware - programs which monitor internet use, causes advert pop-ups and slow a PC's performance.

II. Text Preprocessing

A. Tokenization

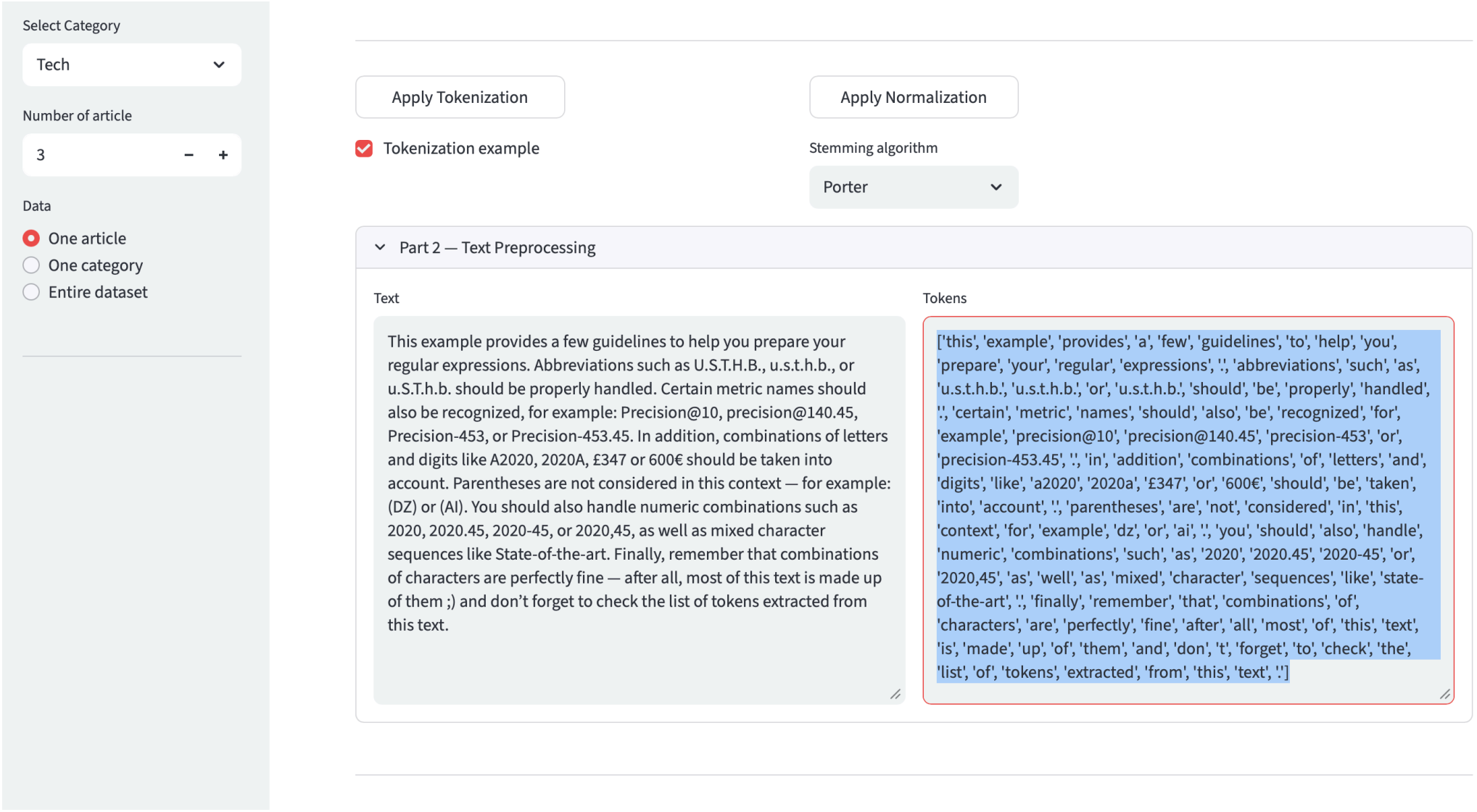
> Implement a tokenization function using '**RegexpTokenizer**' from the NLTK library. Your regular expression must handle the following examples appropriately:

. Before:

"This example provides a few guidelines to help you prepare your regular expressions. Abbreviations such as U.S.T.H.B., u.s.t.h.b., or u.S.T.h.b. should be properly handled. Certain metric names should also be recognized, for example: Precision@10, precision@140.45, Precision-453, or Precision-453.45. In addition, combinations of letters and digits like A2020, 2020A, £347 or 600€ should be taken into account. Parentheses are not considered in this context — for example: (DZ) or (AI). You should also handle numeric combinations such as 2020, 2020.45, 2020-45, or 2020,45, as well as mixed character sequences like State-of-the-art. Finally, remember that combinations of characters are perfectly fine — after all, most of this text is made up of them ;) and don’t forget to check the list of tokens extracted from this text."

. After:

['this', 'example', 'provides', 'a', 'few', 'guidelines', 'to', 'help', 'you', 'prepare', 'your', 'regular', 'expressions', '.', 'abbreviations', 'such', 'as', 'u.s.t.h.b.', 'u.s.t.h.b.', 'or', 'u.s.t.h.b.', 'should', 'be', 'properly', 'handled', '.', 'certain', 'metric', 'names', 'should', 'also', 'be', 'recognized', 'for', 'example', 'precision@10', 'precision@140.45', 'precision-453', 'or', 'precision-453.45', '.', 'in', 'addition', 'combinations', 'of', 'letters', 'and', 'digits', 'like', 'a2020', '2020a', '£347', 'or', '600€', 'should', 'be', 'taken', 'into', 'account', '.', 'parentheses', 'are', 'not', 'considered', 'in', 'this', 'context', 'for', 'example', 'dz', 'or', 'ai', '.', 'you', 'should', 'also', 'handle', 'numeric', 'combinations', 'such', 'as', '2020', '2020.45', '2020-45', 'or', '2020,45', 'as', 'well', 'as', 'mixed', 'character', 'sequences', 'like', 'state-of-the-art', '.', 'finally', 'remember', 'that', 'combinations', 'of', 'characters', 'are', 'perfectly', 'fine', 'after', 'all', 'most', 'of', 'this', 'text', 'is', 'made', 'up', 'of', 'them', 'and', 'don', 't', 'forget', 'to', 'check', 'the', 'list', 'of', 'tokens', 'extracted', 'from', 'this', 'text', '.']


Select Category
Tech
Number of article
3
Data
One article
One category
Entire dataset
Apply Tokenization
Apply Normalization
Tokenization example
Stemming algorithm
Porter
Part 2 — Text Preprocessing
Text
This example provides a few guidelines to help you prepare your regular expressions. Abbreviations such as U.S.T.H.B., u.s.t.h.b., or u.S.T.h.b. should be properly handled. Certain metric names should also be recognized, for example: Precision@10, precision@140.45, Precision-453, or Precision-453.45. In addition, combinations of letters and digits like A2020, 2020A, £347 or 600€ should be taken into account. Parentheses are not considered in this context — for example: (DZ) or (AI). You should also handle numeric combinations such as 2020, 2020.45, 2020-45, or 2020,45, as well as mixed character sequences like State-of-the-art. Finally, remember that combinations of characters are perfectly fine — after all, most of this text is made up of them ;) and don't forget to check the list of tokens extracted from this text.
Tokens
['this', 'example', 'provides', 'a', 'few', 'guidelines', 'to', 'help', 'you', 'prepare', 'your', 'regular', 'expressions', '.', 'abbreviations', 'such', 'as', 'u.s.t.h.b.', 'u.s.t.h.b.', 'or', 'u.s.t.h.b.', 'should', 'be', 'properly', 'handled', '.', 'certain', 'metric', 'names', 'should', 'also', 'be', 'recognized', 'for', 'example', 'precision@10', 'precision@140.45', 'precision-453', 'or', 'precision-453.45', '.', 'in', 'addition', 'combinations', 'of', 'letters', 'and', 'digits', 'like', 'a2020', '2020a', '£347', 'or', '600€', 'should', 'be', 'taken', 'into', 'account', '.', 'parentheses', 'are', 'not', 'considered', 'in', 'this', 'context', 'for', 'example', 'dz', 'or', 'ai', '.', 'you', 'should', 'also', 'handle', 'numeric', 'combinations', 'such', 'as', '2020', '2020.45', '2020-45', 'or', '2020,45', 'as', 'well', 'as', 'mixed', 'character', 'sequences', 'like', 'state-of-the-art', '.', 'finally', 'remember', 'that', 'combinations', 'of', 'characters', 'are', 'perfectly', 'fine', 'after', 'all', 'most', 'of', 'this', 'text', 'is', 'made', 'up', 'of', 'them', 'and', 'don', 't', 'forget', 'to', 'check', 'the', 'list', 'of', 'tokens', 'extracted', 'from', 'this', 'text', '.']

> After tokenization:

. Convert all tokens to lowercase.

. Add the symbols <s> and </s> to indicate the beginning and end of each sentence, respectively.

. Sentence boundaries are identified using the following separators: **?**, **.**, **!**, and newline (**\n**).

. Display the following information:

- The list of tokens.
- The total number of tokens.
- The vocabulary (the set of unique tokens).
- The number of unique tokens.

. The tokenization process must be applicable at three levels: one article, one category and the entire dataset.

Select Category

Tech

Number of article

3 − +

Data

- (•) One article
- ( ) One category
- ( ) Entire dataset

Apply Tokenization

Apply Normalization

☐ Tokenization example

Stemming algorithm

Porter

▾ Part 2 — Text Preprocessing

Tokens

```
<s>
microsoft
seeking
spyware
trojan
</s>
<s>
microsoft
is
investigating
a
trojan
program
that
```

Vocabulary

```
</s>
<s>
a
advert
aims
also
an
and
anti-spyware
anti-virus
as
at
attachment
attack
```

Text

Microsoft seeking spyware trojan

Microsoft is investigating a trojan program that attempts to switch off the firm's anti-spyware software.

The spyware tool was only released by Microsoft in the last few weeks and has been downloaded by six million people. Stephen Toulouse, a security manager at Microsoft, said the malicious program was called Bankash-A Trojan and was being sent as an e-mail attachment. Microsoft said it did not believe the program was widespread and recommended users to use an anti-virus program. The program attempts to disable or delete Microsoft's anti-spyware tool and suppress warning messages given to users.

It may also try to steal online banking passwords or other personal information by

236 Tokens

122 Unique words

B. Stemming (Normalization)

> After tokenization, apply the following stemming algorithms from NLTK:
  . Porter Stemmer
  . Lancaster Stemmer
  . Snowball Stemmer
> For each stemming algorithm, display:
  . The list of tokens before stemming.
  . The list of tokens after stemming.
  . The total number of tokens before and after stemming.
  . The vocabulary after stemming.
  . The number of unique tokens after stemming.
> Apply each stemming algorithm at the same three levels: one article, one category and the entire dataset.

Select Category

Tech

Number of article

3 − +

Data

- (•) One article
- ( ) One category
- ( ) Entire dataset

Apply Tokenization

Apply Normalization

☐ Tokenization example

Stemming algorithm

Porter

▾ Part 2 — Text Preprocessing

Tokens

```
<s>
microsoft
seek
spywar
trojan
</s>
<s>
microsoft
is
investig
a
trojan
program
that
```

```
<s>
microsoft
seeking
spyware
trojan
</s>
<s>
microsoft
is
investigating
a
trojan
program
that
```

236 Tokens normalized

236 Tokens

Vocabulary

```
</s>
<s>
a
advert
aim
also
an
and
anti-spywar
anti-viru
as
at
attach
attack
```

118 Unique words

Text

Microsoft seeking spyware trojan

Microsoft is investigating a trojan program that attempts to switch off the firm's anti-spyware software.

The spyware tool was only released by Microsoft in the last few weeks and has been downloaded by six million people. Stephen Toulouse, a security manager at Microsoft, said the malicious program was called Bankash-A Trojan and was being sent as an e-mail attachment. Microsoft said it did not believe the program was widespread and recommended users to use an anti-virus program.